



API SECURITY RISK ANALYSIS REPORT

Cyber Security Internship — Task 3

PREPARED BY	Shantan Narayndas
CIN ID	FIT/APR26/CS8031
TRACK	FUTURE_CS_03
ASSESSMENT DATE	May 4, 2026
TOOL USED	Postman API Client
API TESTED	JSONPlaceholder REST API
TEST TYPE	Manual API Security Testing

FUTURE_CS_03 | CONFIDENTIAL | MAY 2026

SECTION 1

Executive Summary

An API Security Risk Analysis was conducted on the **JSONPlaceholder REST API** (<https://jsonplaceholder.typicode.com>) using **Postman API Client**. A total of **6 security tests** were performed manually to identify common API vulnerabilities defined in the OWASP API Security Top 10.

The analysis revealed **4 high-risk vulnerabilities**, 1 medium-risk issue, and 1 informational finding. The most critical issues found were complete absence of authentication for all API operations and missing security headers.



#	TEST PERFORMED	METHOD	STATUS CODE	RISK LEVEL	RESULT
1	Unauthenticated Data Access	GET / users	200 OK	High	VULNERABLE
2	Missing Security Headers	GET / users	200 OK	High	VULNERABLE
3	Unauthorized Data Creation	POST / posts	201 Created	High	VULNERABLE
4	Unauthorized Data Deletion	DELETE /posts/1	200 OK	High	VULNERABLE
5	Error Handling Check	GET / users/ 9999	404	Medium	WEAK
6	HTTP vs HTTPS Enforcement	GET / users (HTTP)	200 OK	Low	VULNERABLE

SECTION 2

What is API Security Testing?

What is an API?

An API (Application Programming Interface) is a way for software applications to talk to each other. For example, when you use a weather app on your phone, the app sends a request to a weather server's API and gets back weather data. APIs are everywhere — banking apps, social media, shopping websites all use APIs.

What is API Security Testing?

API Security Testing checks whether an API is properly secured against attacks. It examines whether attackers can access data they shouldn't, modify or delete records without permission, or bypass authentication and authorization controls. We use Postman to send different types of requests to the API and observe how it responds.

TOOL USED — POSTMAN

Postman is the world's most popular API testing tool used by developers and security professionals. It allows users to send HTTP requests (GET, POST, PUT, DELETE) to any API and inspect the responses. During this assessment, Postman was used to manually test the JSONPlaceholder API for common security vulnerabilities.

API TESTED — JSONPLACEHOLDER

API Name	JSONPlaceholder REST API
Base URL	https://jsonplaceholder.typicode.com
Type	Public REST API (for testing purposes)
Endpoints Tested	/users, /posts, /posts/1, /users/9999
HTTP Methods Used	GET, POST, DELETE
Authentication Required	None (as found during testing)

HTTP METHODS EXPLAINED

METHOD	WHAT IT DOES	REAL-WORLD EXAMPLE
GET	Retrieve / Read data	View your profile on a website
POST	Create new data	Submit a registration form
PUT	Update existing data	Edit your account settings
DELETE	Delete data	Remove a post or account

Detailed Security Findings

HIGH

Finding #1 — Unauthenticated Data Access (Broken Authentication)

REQUEST

GET https://jsonplaceholder.typicode.com/users

RESPONSE CODE

200 OK

OWASP CATEGORY

API2:2023 Broken Authentication

DATA EXPOSED

Names, Emails, Phone Numbers, Addresses, Companies

Sending a GET request to /users returned complete user profiles for all 10 users without requiring any authentication token, API key, or login credentials. This means any person on the internet can access all user data freely. In a real-world scenario, this would be a critical data breach exposing personally identifiable information (PII) of all users.

Fix: Implement token-based authentication (JWT or API keys). All endpoints that return user data must require a valid Authorization header: Authorization: Bearer <token>

HIGH

Finding #2 — Missing Critical Security Headers

REQUEST

GET /users — Response Headers

OWASP CATEGORY

API8:2023 Security Misconfiguration

HEADER	STATUS	RISK IF MISSING
X-Frame-Options	MISSING	Clickjacking attacks possible
Content-Security-Policy	MISSING	XSS attacks possible
Strict-Transport-Security	MISSING	HTTPS downgrade attacks
X-Content-Type-Options	PRESENT	MIME sniffing protected
X-Powered-By: Express	WARNING	Reveals server technology to attackers

Fix: Add X-Frame-Options: DENY, Content-Security-Policy, and Strict-Transport-Security headers to all API responses. Remove X-Powered-By header.

Detailed Security Findings (Continued)

HIGH

Finding #3 — Unauthorized Data Creation (Broken Object Level Authorization)

REQUEST

POST https://jsonplaceholder.typicode.com/posts

RESPONSE CODE

201 Created

OWASP CATEGORY

API1:2023 Broken Object Level Authorization

AUTHENTICATION USED

None

A POST request was sent with a JSON body containing a fake post titled "Hacked Post — This was created without any authentication." The API responded with 201 Created, confirming the record was successfully added. No API key, token, or credentials were provided. In a real API, this would allow any anonymous user to inject malicious content, spam, or false records into the system.

Request Body Sent:

```
{"title": "Hacked Post", "body": "Created without authentication", "userId": 1}
```

Response: 201 Created — Record accepted with no authentication required!

Fix: Require authentication tokens for all POST, PUT, and DELETE operations. Validate that the requesting user has permission to create records.

HIGH

Finding #4 — Unauthorized Data Deletion (Broken Object Level Authorization)

REQUEST

DELETE https://jsonplaceholder.typicode.com/posts/1

RESPONSE CODE

200 OK

OWASP CATEGORY

API1:2023 Broken Object Level Authorization

AUTHENTICATION USED

None

A DELETE request was sent to remove post #1. The API returned 200 OK confirming the deletion — with absolutely no authentication or authorization check. Any malicious actor could delete any record from the API database by simply knowing its ID number. In a real banking or e-commerce API, this could allow attackers to delete transactions, orders, or user accounts.

Fix: Implement strict authorization checks before allowing DELETE operations. Only the record owner or admin should be able to delete data. Return 401 Unauthorized or 403 Forbidden for unauthenticated requests.

MEDIUM Finding #5 — Poor Error Handling (Information Leakage)

REQUEST

GET https://jsonplaceholder.typicode.com/users/9999

RESPONSE CODE

404 Not Found

When requesting a non-existent user (ID: 9999), the API returned a 404 status code with an empty response body `{}`. While returning 404 is correct behavior, the empty body provides no meaningful error message. A well-designed API should return a structured error response with an error code and description to help developers understand what went wrong.

Fix: Return structured error responses — e.g., `{"error": "User not found", "code": 404, "message": "The requested resource does not exist"}`

LOW Finding #6 — No HTTPS Enforcement (Insecure Transport)

REQUEST

GET http://jsonplaceholder.typicode.com/users

RESPONSE CODE

200 OK (No Redirect)

When the API was accessed over HTTP (insecure) instead of HTTPS (secure), it returned 200 OK without redirecting to HTTPS. This means data can travel over an unencrypted connection — especially dangerous on public WiFi networks where attackers can intercept the traffic (Man-in-the-Middle attack).

Fix: Force HTTPS redirect for all HTTP requests. Add Strict-Transport-Security header: `Strict-Transport-Security: max-age=31536000; includeSubDomains`

SECTION 4

OWASP API Security Top 10 Mapping

The **OWASP API Security Top 10** is a globally recognized list of the most critical API security risks, maintained by the Open Web Application Security Project (OWASP). The vulnerabilities found in this assessment map directly to the following OWASP categories:

API1

Broken Object Level Authorization

APIs expose endpoints that handle object identifiers without checking if the user has permission to access or modify those objects.

⚠ **FOUND — FINDINGS #3 AND #4**

API2

Broken Authentication

Authentication mechanisms are implemented incorrectly or missing entirely, allowing attackers to access resources without credentials.

⚠ **FOUND — FINDING #1**

API3

Broken Object Property Level Authorization

API returns more data than needed, exposing sensitive fields that should be hidden from certain users.

⚠ **FOUND — FINDING #1 (FULL USER DATA EXPOSURE)**

API5

Broken Function Level Authorization

APIs fail to restrict access to sensitive functions like DELETE or admin operations based on user roles.

⚠ **FOUND — FINDING #4**

API7

Server Side Request Forgery

API fetches a remote resource without validating the user-supplied URL, potentially exposing internal services.

Not tested in this assessment.

API8

Security Misconfiguration

Missing security headers, insecure defaults, unnecessary HTTP methods enabled, or sensitive information exposed in headers.

⚠ **FOUND — FINDINGS #2 AND #6**

RISK SUMMARY TABLE

#	VULNERABILITY	RISK LEVEL	OWASP REFERENCE	PRIORITY
1	Unauthenticated Data Access	High	API2 Broken Authentication	Fix Immediately

#	VULNERABILITY	RISK LEVEL	OWASP REFERENCE	PRIORITY
	Missing Security Headers	High	API8 Security Misconfiguration	Fix Immediately
3	Unauthorized Data Creation	High	API11 Broken Object Authorization	Fix Immediately
4	Unauthorized Data Deletion	High	API1 + API5 Authorization	Fix Immediately
5	Poor Error Handling	Medium	API8 Misconfiguration	Fix Soon
6	No HTTPS Enforcement	Low	API8 Misconfiguration	Fix When Possible

Recommendations & Conclusion

TOP RECOMMENDATIONS

1

Implement Authentication on All Endpoints

Every API endpoint that returns or modifies data must require a valid authentication token. Use JWT (JSON Web Token) or OAuth 2.0. Unauthenticated requests should receive 401 Unauthorized.

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

2

Implement Role-Based Authorization (RBAC)

Not every authenticated user should be able to delete or modify data. Implement role checks so only admins or the data owner can perform write/delete operations. Return 403 Forbidden when a user lacks permission.

3

Add All Missing Security Headers

Configure the web server to include these headers in every API response:

```
X-Frame-Options: DENY
Content-Security-Policy: default-src 'self'
Strict-Transport-Security: max-age=31536000
X-Content-Type-Options: nosniff
```

4

Force HTTPS — Redirect All HTTP Traffic

Configure the server to automatically redirect all HTTP requests to HTTPS. Never allow sensitive data to travel over unencrypted connections.

```
HTTP 301 Redirect → https://api.example.com
```

5

Improve Error Handling — Structured Error Responses

Return meaningful, consistent error messages without revealing internal system details. Use a standard error format across all endpoints.

```
{"error": "Not Found", "code": 404, "message": "User does not exist"}
```

CONCLUSION

The API security assessment of **JSONPlaceholder REST API** using Postman revealed **4 high-risk vulnerabilities**, 1 medium-risk issue, and 1 low-risk finding across 6 security tests performed. The most critical finding is the complete absence of authentication — allowing any anonymous user to read all user data, create new records, and delete existing records without any credentials.

These vulnerabilities align with the **OWASP API Security Top 10**, particularly Broken Authentication (API2) and Broken Object Level Authorization (API1). In a real-world production API handling financial transactions, medical records, or personal data, these vulnerabilities would constitute a severe security breach.

The primary recommendation is to implement proper authentication (JWT/OAuth 2.0) and role-based authorization immediately, followed by adding security headers and enforcing HTTPS. Regular API security testing using tools like Postman should be incorporated into the software development lifecycle to catch vulnerabilities before production deployment.

Disclaimer: This assessment was performed on a public test API (JSONPlaceholder) designed for educational and testing purposes. No real user data was accessed or harmed. This report was prepared as part of the Future Interns Cyber Security Internship — FUTURE_CS_03.

Report prepared by: Shantan Narayndas | CIN: FIT/APR26/CS8031 | Tool: Postman | Date: May 4, 2026